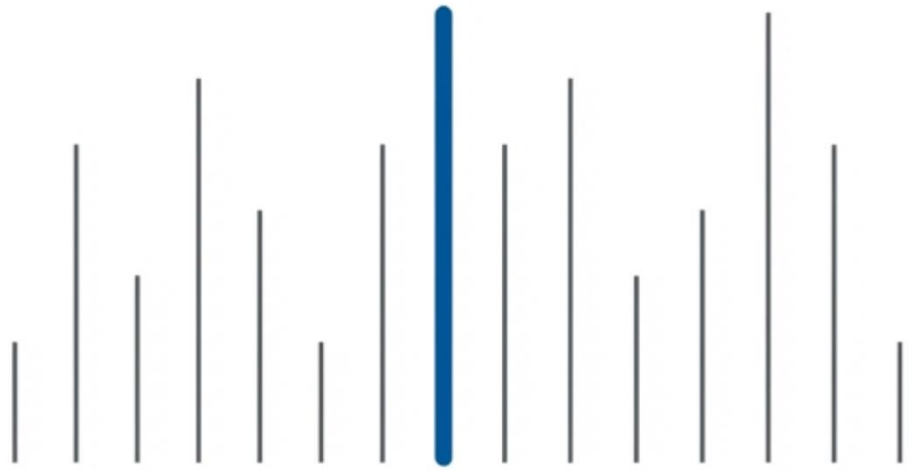
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Algoritmos de ordenamiento:

Mergesort y quicksort



Objetivos

- Descomponer el paradigma divide y vencerás en sus 3 etapas fundamentales: Dividir, conquistar y combinar aplicándolas a un problema de ordenamiento.

- Descomponer el paradigma de *Divide y Vencerás* en sus tres etapas fundamentales: Dividir, Conquistar y Combinar, aplicándolas a los problemas de ordenamiento.

- Demostrar que Mergesort garantiza una complejidad temporal estricta de $\Theta(n \log n)$, e identificar el compromiso (*trade-off*) subyacente de complejidad espacial $\Theta(n)$ debido a la necesidad de memoria auxiliar.

- Revisar el algoritmo de Quicksort entendiendo el proceso de partición de arreglos (*Partitioning*), analizando su complejidad temporal esperada de $\mathcal{O}(n \log n)$ y evaluando matemáticamente su degradación al peor caso $\mathcal{O}(n^2)$.

Motivación

La transición de algoritmos de ordenamiento cuadráticos $\mathcal{O}(n^2)$ a logarítmico-lineales $\mathcal{O}(n \log n)$ no es una mera curiosidad matemática; es una exigencia absoluta en la ingeniería de software escalable. A medida que el volumen de datos crece, la disparidad en el tiempo de ejecución entre estas dos clases de complejidad se vuelve insalvable para cualquier arquitectura de hardware actual.

std::sort

Paradigma Divide y vencerás

Dividir:

Separar el problema en subproblemas menores.

Conquistar:

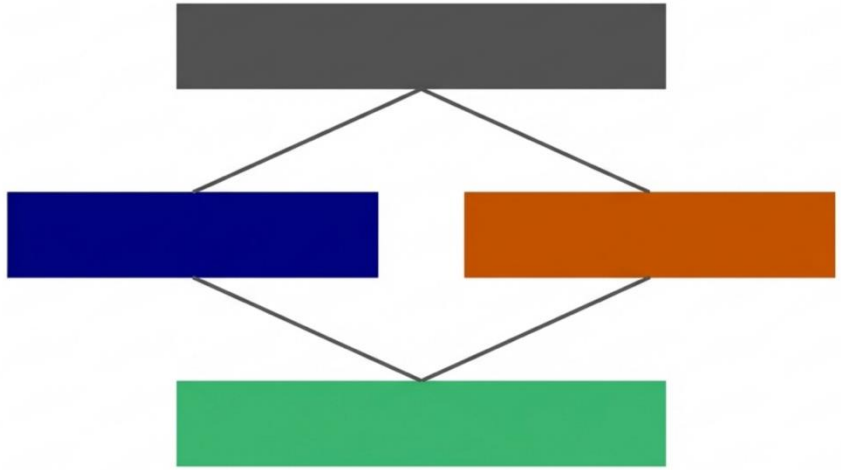
Resolver los subproblemas de forma recursiva.

Combinar:

Unir las soluciones parciales para formar la solución final.

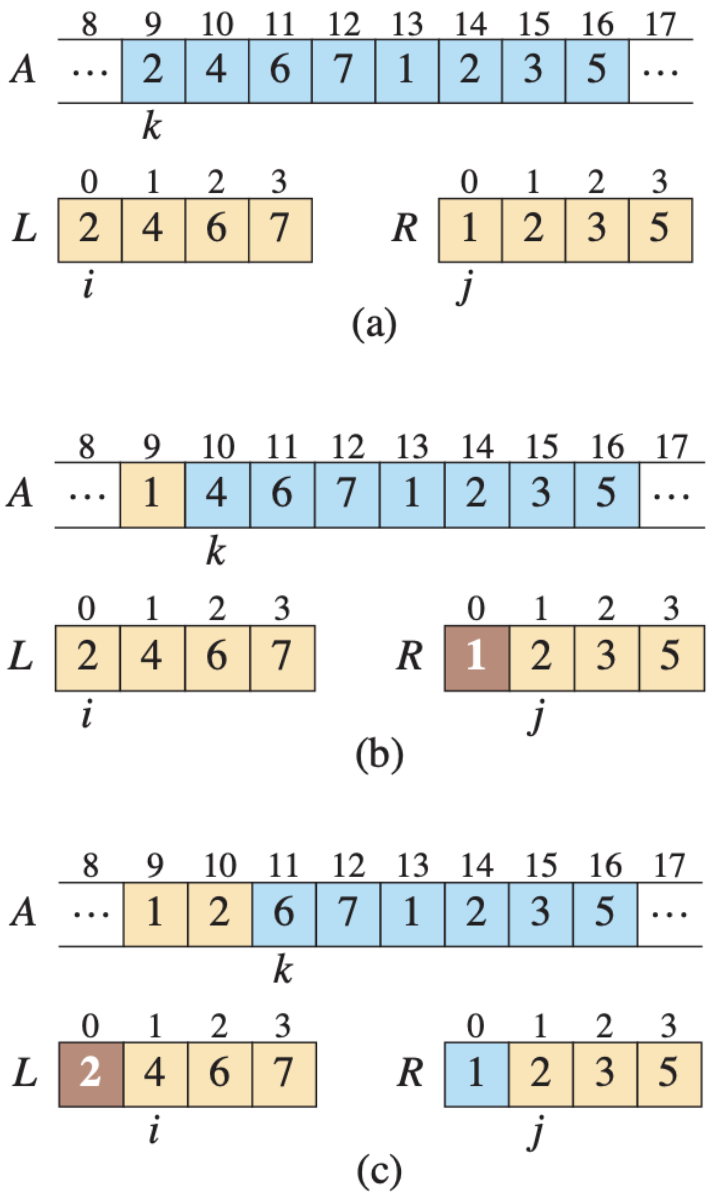
Mergesort

```
MERGE-SORT(A,p,r)
if p >= r
    return
q = ⌊(p + r) / 2⌋
MERGE-SORT(A, p, q)
MERGE-SORT(A, q + 1, r)
MERGE(A, p, q, r)
```

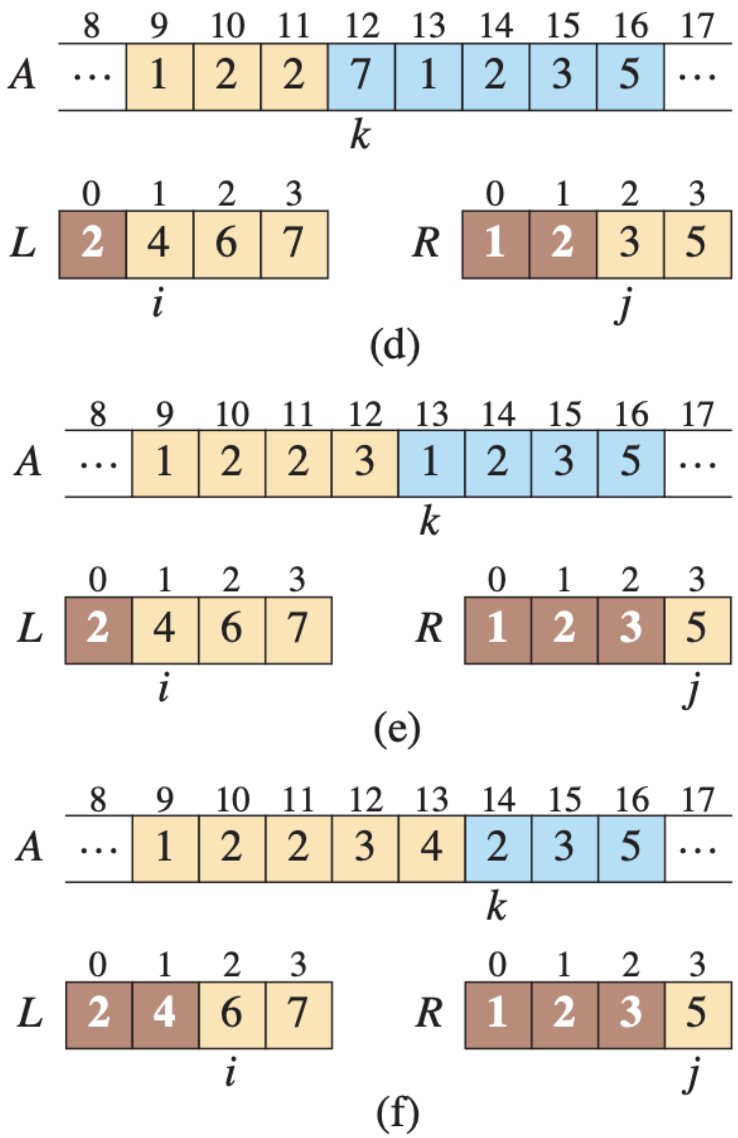


El núcleo de la carga computacional recae en el subrutina MERGE. Se asume que los subarreglos $A[p:q]$ y $A[q+1:r]$ ya se encuentran ordenados. El subrutina crea arreglos dinámicos locales L y R, copia el contenido y fusiona ambos bloques sobre escribiendo la memoria original de $A[p:r]$.

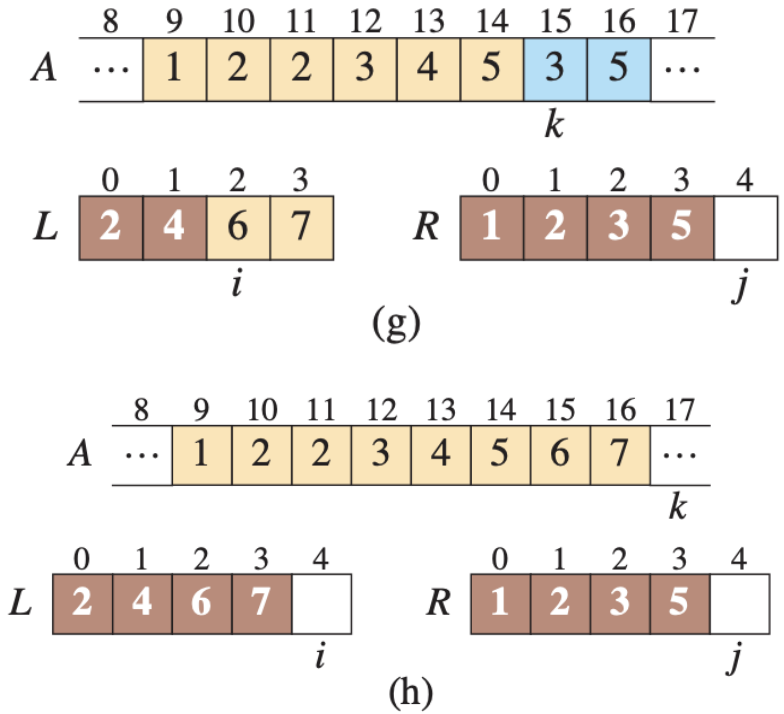
```
MERGE(A, p, q, r)
n_L = q - p + 1                 // Longitud del subarreglo izquierdo
n_R = r - q                     // Longitud del subarreglo derecho
let L[0 : n_L - 1] and R[0 : n_R - 1] be new arrays
for i = 0 to n_L - 1            // Copia al arreglo L
    L[i] = A[p + i]
for j = 0 to n_R - 1            // Copia al arreglo R
    R[j] = A[q + 1 + j]
i = 0
j = 0
k = p
while i < n_L and j < n_R       // Fusión principal
    if L[i] <= R[j]
        A[k] = L[i]
        i = i + 1
    else
        A[k] = R[j]
        j = j + 1
    k = k + 1
while i < n_L                   // Volcado residual de L (si aplica)
    A[k] = L[i]
    i = i + 1
    k = k + 1
while j < n_R                   // Volcado residual de R (si aplica)
    A[k] = R[j]
    j = j + 1
    k = k + 1
```



```
MERGE(A, p, q, r)
n_L = q - p + 1                 // Longitud del subarreglo izquierdo
n_R = r - q                     // Longitud del subarreglo derecho
let L[0 : n_L - 1] and R[0 : n_R - 1] be new arrays
for i = 0 to n_L - 1            // Copia al arreglo L
    L[i] = A[p + i]
for j = 0 to n_R - 1            // Copia al arreglo R
    R[j] = A[q + 1 + j]
i = 0
j = 0
k = p
while i < n_L and j < n_R       // Fusión principal
    if L[i] <= R[j]
        A[k] = L[i]
        i = i + 1
    else
        A[k] = R[j]
        j = j + 1
    k = k + 1
while i < n_L                   // Volcado residual de L (si aplica)
    A[k] = L[i]
    i = i + 1
    k = k + 1
while j < n_R                   // Volcado residual de R (si aplica)
    A[k] = R[j]
    j = j + 1
    k = k + 1
```



```
MERGE(A, p, q, r)
n_L = q - p + 1                 // Longitud del subarreglo izquierdo
n_R = r - q                     // Longitud del subarreglo derecho
let L[0 : n_L - 1] and R[0 : n_R - 1] be new arrays
for i = 0 to n_L - 1            // Copia al arreglo L
    L[i] = A[p + i]
for j = 0 to n_R - 1            // Copia al arreglo R
    R[j] = A[q + 1 + j]
i = 0
j = 0
k = p
while i < n_L and j < n_R       // Fusión principal
    if L[i] <= R[j]
        A[k] = L[i]
        i = i + 1
    else
        A[k] = R[j]
        j = j + 1
    k = k + 1
while i < n_L                   // Volcado residual de L (si aplica)
    A[k] = L[i]
    i = i + 1
    k = k + 1
while j < n_R                   // Volcado residual de R (si aplica)
    A[k] = R[j]
    j = j + 1
    k = k + 1
```



Mergesort

```
MERGE-SORT(A,p,r)
if p >= r
    return
q = ⌊(p + r) / 2⌋
MERGE-SORT(A, p, q)
MERGE-SORT(A, q + 1, r)
MERGE(A, p, q, r)
```

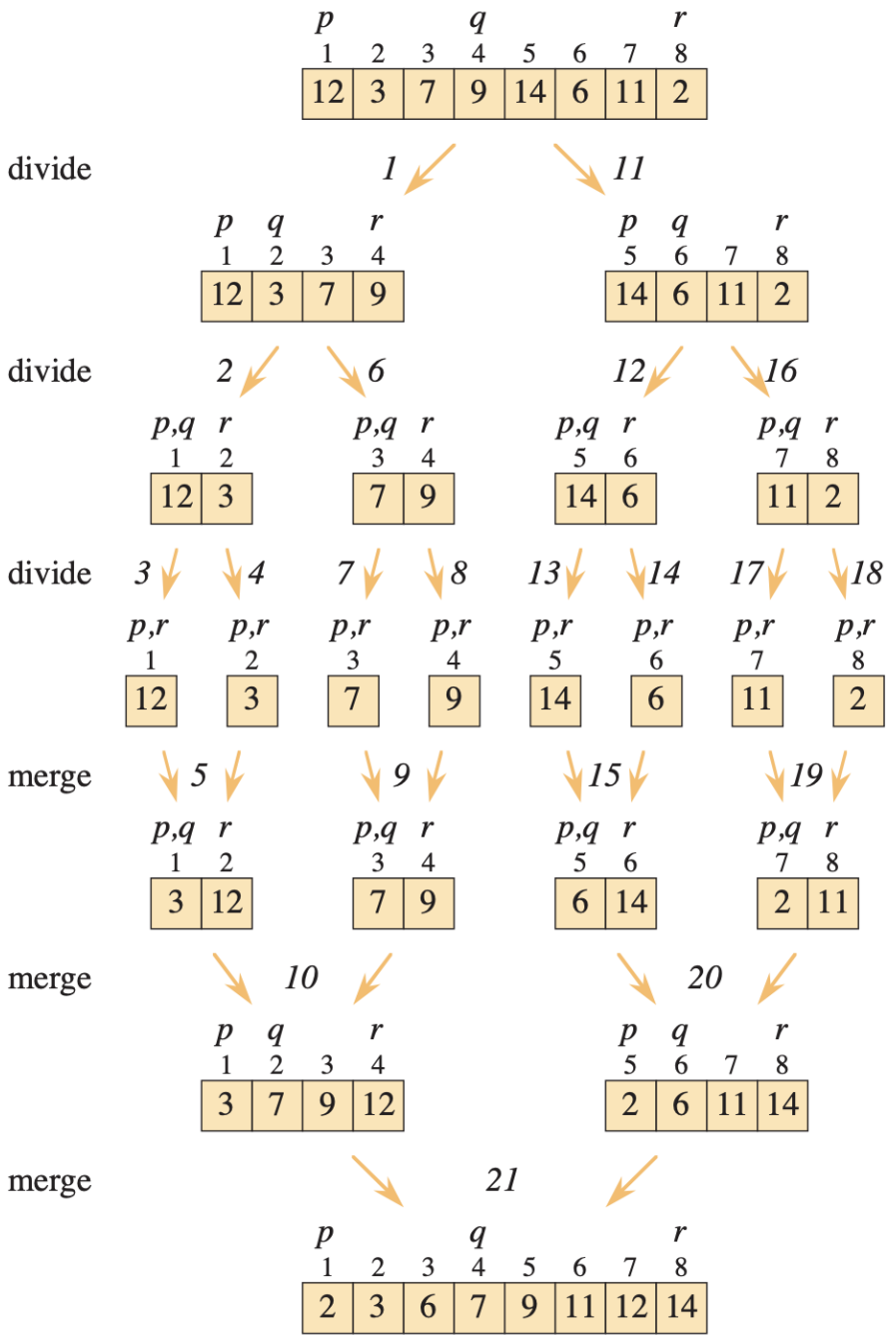
División ($\boldsymbol{D(n)}$): El cálculo del punto medio es una simple operación aritmética que toma tiempo constante, $\Theta(1)$.

Conquista ($\boldsymbol{aT(n/b)}$): Se efectúan 2 llamadas recursivas sobre subarreglos de tamaño $n/2$, lo cual impone un costo de $2T(n/2)$.

Combinación ($\boldsymbol{C(n)}$): La subrutina MERGE itera sobre un total de n elementos a través de tres ciclos while, ejecutando asignaciones simples que consumen tiempo lineal, por lo que $C(n) = \Theta(n)$.

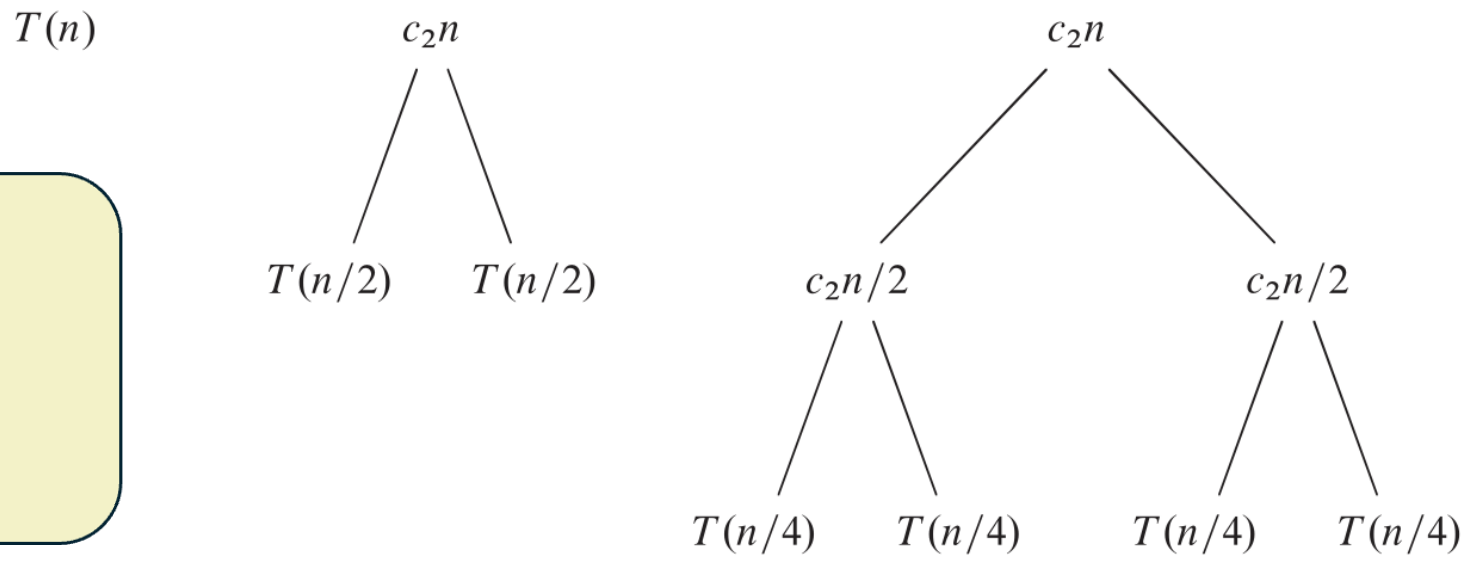
Mergesort

```
MERGE-SORT(A,p,r)
if p >= r
    return
q = ⌊(p + r) / 2⌋
MERGE-SORT(A, p, q)
MERGE-SORT(A, q + 1, r)
MERGE(A, p, q, r)
```



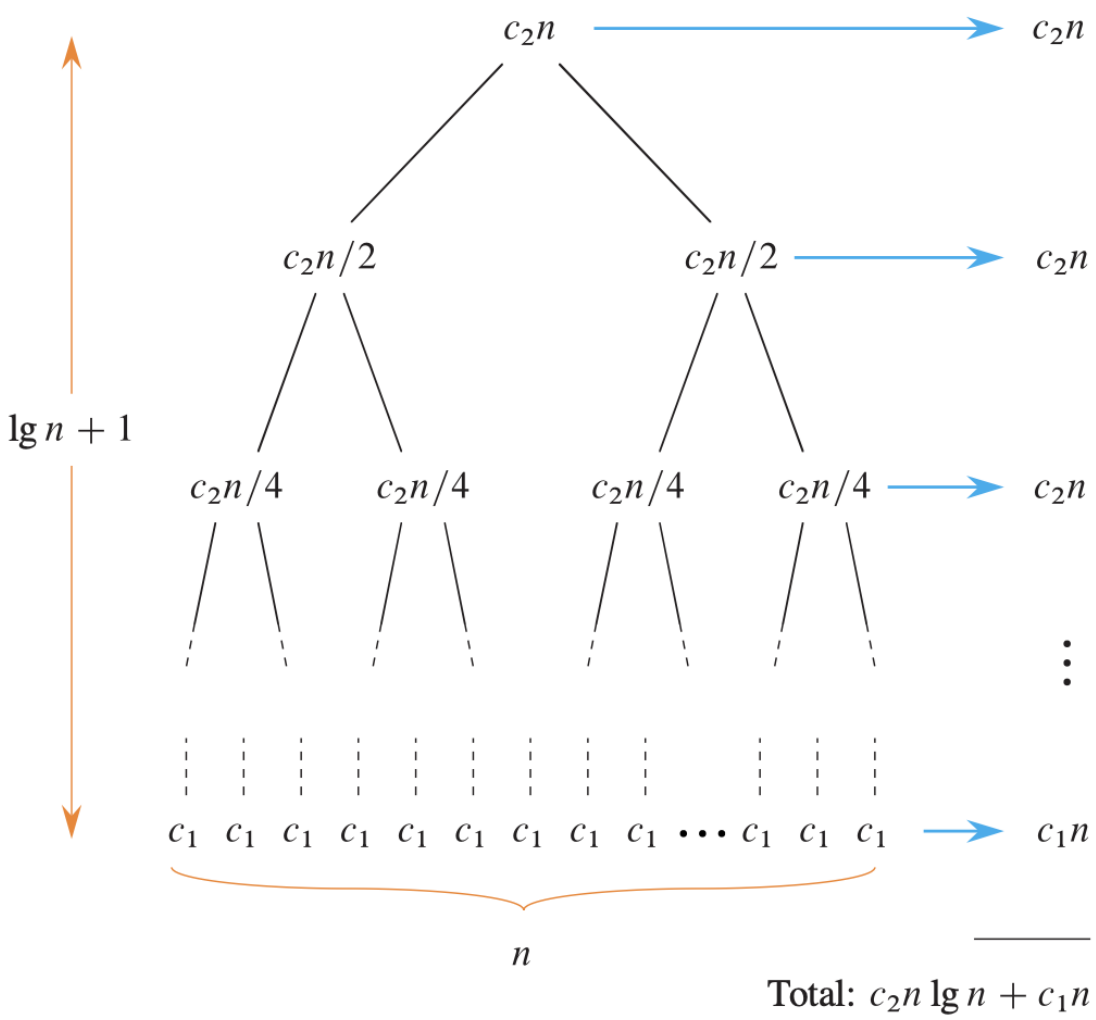
Mergesort

```
MERGE-SORT(A,p,r)
if p >= r
    return
q = ⌊(p + r) / 2⌋
MERGE-SORT(A, p, q)
MERGE-SORT(A, q + 1, r)
MERGE(A, p, q, r)
```



Mergesort

```
MERGE-SORT(A,p,r)
if p >= r
    return
q = ⌊(p + r) / 2⌋
MERGE-SORT(A, p, q)
MERGE-SORT(A, q + 1, r)
MERGE(A, p, q, r)
```



Quiksort

ALGORITHM 63
PARTITION
C. A. R. HOARE
Elliott Brothers Ltd., Borehamwood, Hertfordshire, Eng.

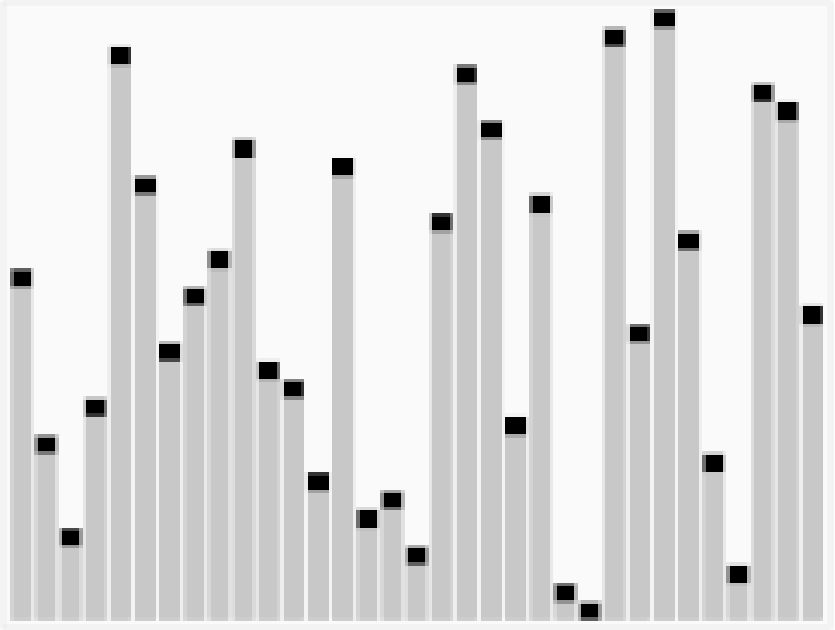
```
procedure  partition (A,M,N,I,J);  value M,N;
             array A;  integer M,N,I,J;

comment  I and J are output variables, and A is the array (with
subscript bounds M:N) which is operated upon by this procedure.
Partition takes the value X of a random element of the array A,
and rearranges the values of the elements of the array in such a
way that there exist integers I and J with the following properties:
            M ≦ J < I ≦ N provided M < N
            A[R] ≦ X for M ≦ R ≦ J
            A[R] = X for J < R < I
            A[R] ≧ X for I ≦ R ≦ N
The procedure uses an integer procedure random (M,N) which
chooses equiprobably a random integer F between M and N, and
also a procedure exchange, which exchanges the values of its two
parameters;
begin    real X;  integer F;
         F := random (M,N);  X := A[F];
         I := M;  J := N;
up:      for I := I step 1 until N do
                    if X < A [I] then go to down;
         I := N;
down:    for J := J  step −1 until M do
                    if A[J]<X then go to change;
         J := M;
change:  if I < J then begin exchange (A[I], A[J]);
                              I := I + 1; J := J − 1;
                              go to up
                        end
else     if I < F then begin exchange (A[I], A[F]);
                              I := I + 1
                        end
else     if F < J then  begin exchange (A[F], A[J]);
                              J := J − 1
                        end;
end      partition
```

ALGORITHM 64
QUICKSORT
C. A. R. HOARE
Elliott Brothers Ltd., Borehamwood, Hertfordshire, Eng.

```
procedure   quicksort (A,M,N);  value M,N;
              array A;  integer M,N;
comment  Quicksort is a very fast and convenient method of
sorting an array in the random-access store of a computer. The
entire contents of the store may be sorted, since no extra space is
required. The average number of comparisons made is 2(M−N) ln
(N−M), and the average number of exchanges is one sixth this
amount. Suitable refinements of this method will be desirable for
its implementation on any actual computer;
begin       integer I,J;
            if M < N then begin partition (A,M,N,I,J);
                                 quicksort (A,M,J);
                                 quicksort (A, I, N)
                             end
end         quicksort
```

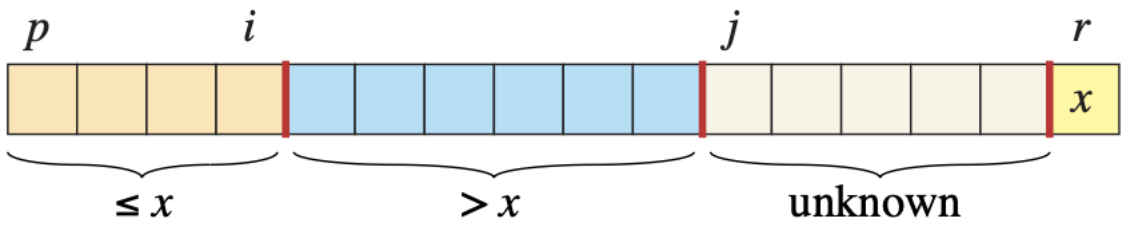
Quicksort

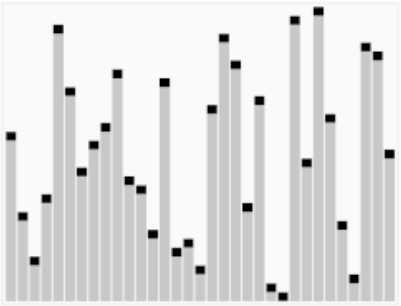
```
QUICKSORT(A, p, r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q - 1)
    QUICKSORT(A, q + 1, r)
```



Quicksort

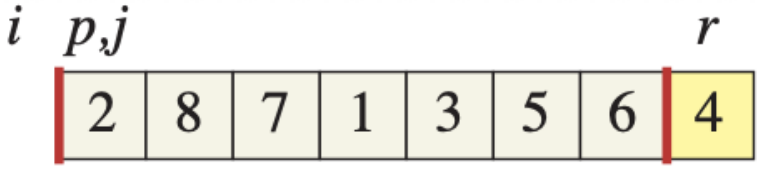
```
PARTITION(A, p, r)
x = A[r]        // El pivote se elige como el último elemento
i = p - 1       // Índice del elemento más pequeño
for j = p to r - 1
    if A[j] <= x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```





Quicksort

```
PARTITION(A, p, r)
1 x = A[r]        // El pivote se elige como el último elemento
2 i = p - 1       // Índice del elemento más pequeño
3 for j = p to r - 1
4     if A[j] <= x
5         i = i + 1
6         exchange A[i] with A[j]
7 exchange A[i + 1] with A[r]
8 return i + 1
```



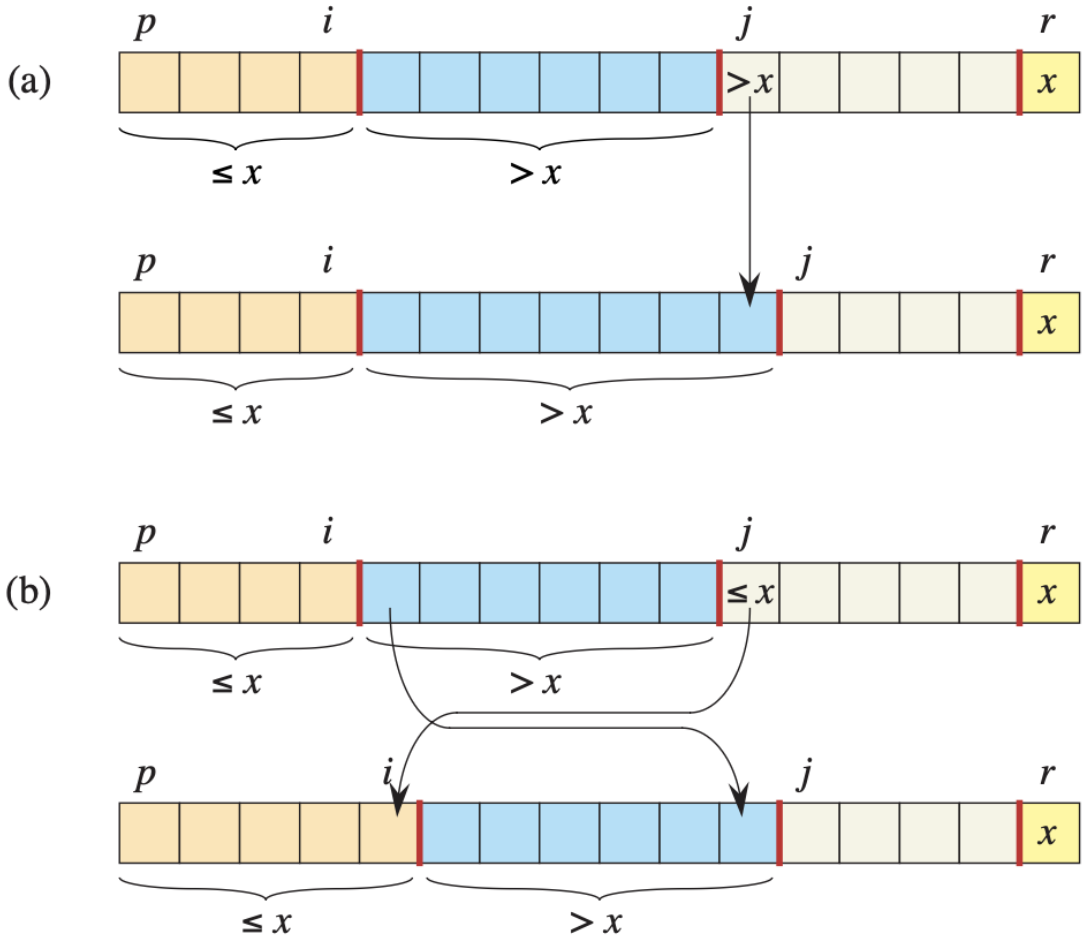
Inicialización: Antes de la primera iteración del bucle, tenemos $i = p - 1$ y $j = p$. Debido a que no hay valores entre p e i, y no hay valores entre $i + 1$ y $j - 1$, las dos primeras condiciones del invariante de bucle se satisfacen de manera trivial. La asignación en la línea 1 satisface la tercera condición.

Quicksort

Mantenimiento: consideramos dos casos, dependiendo del resultado de la evaluación en la línea 4.

- (a) ***muestra lo que sucede cuando*** $\boldsymbol{A[j] > x}$***:*** la única acción dentro del bucle es incrementar j. Después de que j ha sido incrementado, la segunda condición se mantiene para $A[j-1]$ y todas las demás entradas permanecen inalteradas.
- (b) ***muestra lo que sucede cuando*** $\boldsymbol{A[j] \leq x}$***:*** el bucle incrementa i, intercambia $A[i]$ con $A[j]$, y luego incrementa j. Debido al intercambio, ahora tenemos que $A[i] \leq x$, satisfaciendo la condición 1. De manera similar, también tenemos que $A[j-1] > x$, ya que el elemento que fue reubicado en $A[j-1]$ mediante el intercambio es, por definición del invariante de bucle, mayor que x.

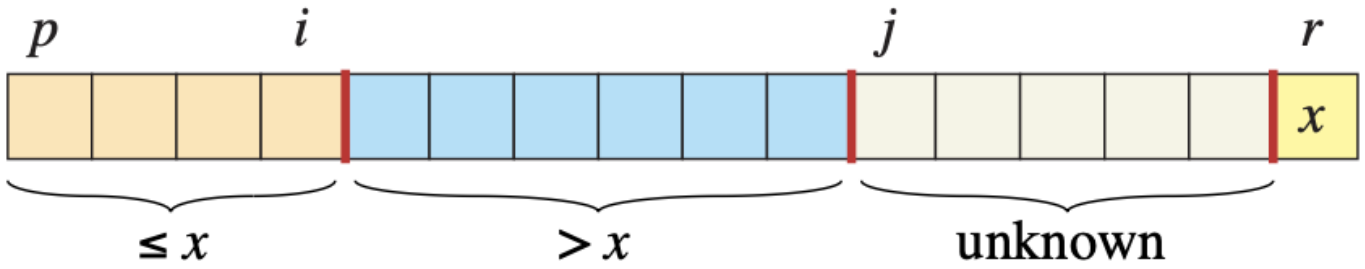
```
PARTITION(A, p, r)
1 x = A[r]          // El pivote se elige como el último elemento
2 i = p - 1         // Índice del elemento más pequeño
3 for j = p to r - 1
4     if A[j] <= x
5         i = i + 1
6         exchange A[i] with A[j]
7 exchange A[i + 1] with A[r]
8 return i + 1
```



Quicksort

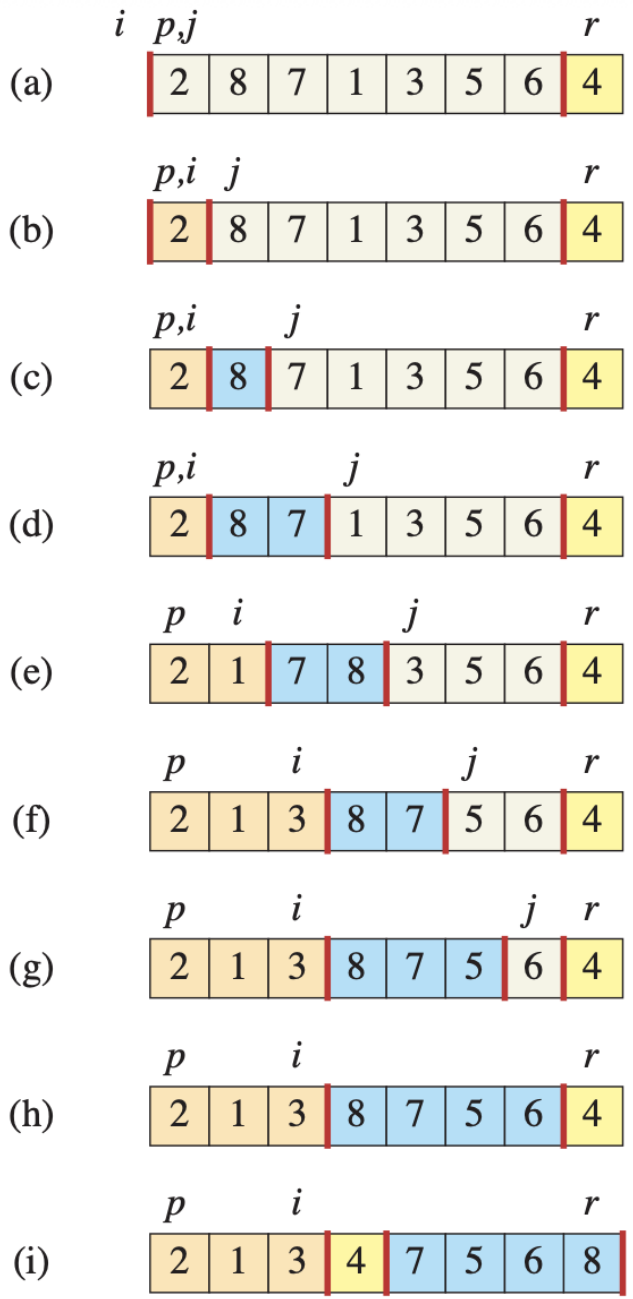
Terminación: Dado que el bucle realiza exactamente $r - p$ iteraciones, este termina cuando $j = r$. En ese punto, el subarreglo no examinado $A[j{:}r-1]$ se encuentra vacío, y cada entrada en el arreglo pertenece a uno de los otros tres conjuntos descritos por el invariante. En consecuencia, los valores en el arreglo han sido particionados en tres conjuntos: aquellos menores o iguales a x (el lado inferior), aquellos mayores que x (el lado superior), y un conjunto unitario que contiene a x (el pivote).

```
PARTITION(A, p, r)
x = A[r]          // El pivote se elige como el último elemento
i = p - 1         // Índice del elemento más pequeño
for j = p to r - 1
    if A[j] <= x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



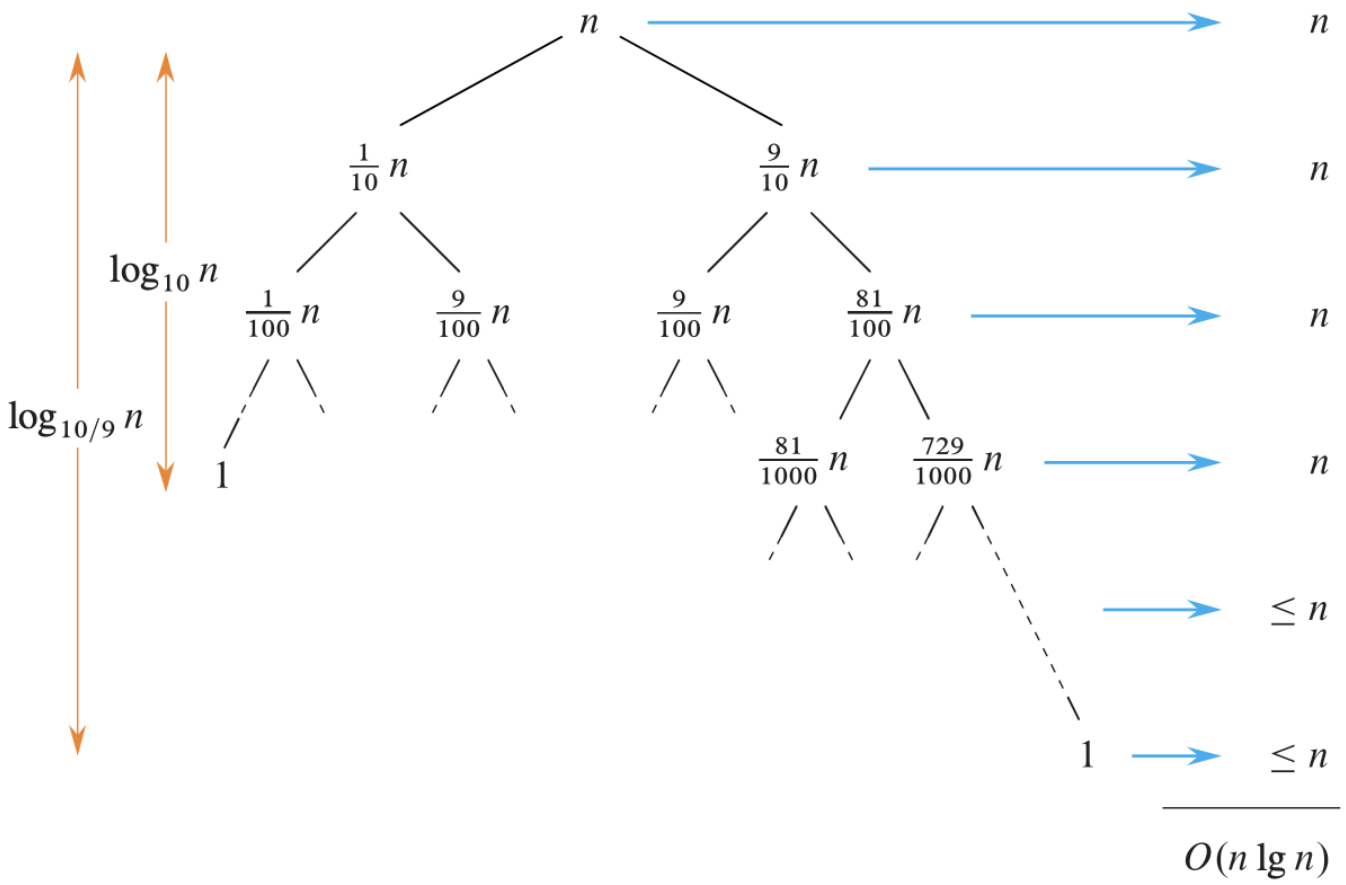
Quicksort

```
PARTITION(A, p, r)
x = A[r]          // El pivote se elige como el último elemento
i = p - 1         // Índice del elemento más pequeño
for j = p to r - 1
    if A[j] <= x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i + 1] with A[r]
return i + 1
```



Quicksort

Supongamos, por ejemplo, que el algoritmo de partición siempre produce una división proporcional de 9 a 1, lo cual a primera vista parece bastante desbalanceado. Obtenemos entonces la siguiente recurrencia

$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$

para el tiempo de ejecución de Quick Sort. La Figura muestra el árbol de recursión para esta recurrencia, donde, por simplicidad, la función impulsora $\Theta(n)$ ha sido reemplazada por n, lo cual no afectará la solución asintótica de la recurrencia.